AgriRent — Service Design Benchmark & Architecture Specification

Course: CS543: Web Services (Assignment 2 of 14)
Student / Team Member: Akshay Kumar
System Name: AgriRent (Peer-to-Peer Agricultural Equipment Marketplace)

Task 1: System Capabilities (The Verbs)

From the AgriRent Assignment 1 system brief, the platform provides the following distinct capabilities:

1. **VerifyIdentity (KYC Verification):** Authenticates user credentials and validates government/farmer identification records prior to machine transactions.
2. **PublishListing (Equipment Management):** Enables owners to list farming equipment with technical specs, daily rental prices, security deposit requirements, and late fine parameters.
3. **DiscoverEquipment (Search & Availability):** Queries listings based on equipment category, geographic location radius, and real-time date availability without schedule overlaps.
4. **ReserveRental (Booking Lifecycle):** Handles the rental reservation process, generates price breakdowns (including 3% platform fee), and enforces a mandatory 24-hour owner response SLA.
5. **ProcessPayment (Financial Hold & Settlement):** Interfaces with payment providers to authorize advance rental charges, hold security deposits in escrow, and execute post-rental payouts or deductions.
6. **ValidateHandoverReturn (PIN Verification):** Manages physical handovers and returns using system-generated, single-use PIN tokens exchanged between renter and equipment owner.
7. **ArbitrateDispute (Damage & Claim Handling):** Captures photographic evidence and claims for damaged machinery and manages administrative resolutions for contested deposit deductions.

Task 2: Service Design Benchmark Overview

- **Services Identified:** User Service, Catalogue Service, Booking Service (Central Coordinator), Payment Service.
- **Diagram Files:** `services.drawio` and `services.png`
- **Data Isolation Rule:** Each microservice strictly owns its operational data store. No shared tables or direct inter-service database queries are permitted.

Task 3: Service Contracts (External Interfaces)

Contracts expose business operations while completely abstracting internal table names, SQL queries, and storage implementation details.

1. Booking Service (Main Service 1)

Operation Name	Input	Output	Errors
createBookingRequest	renterId, equipmentId, startDate, endDate	bookingId, estimatedCost, status: PENDING	EQUIPMENT_UNAVAILABLE, USER_NOT_VERIFIED, INVALID_DATES
respondBooking	bookingId, ownerId, action (APPROVE/REJECT)	bookingId, newStatus, paymentDeadline	UNAUTHORIZED_OWNER, REQUEST_EXPIRED, ALREADY_HANDLED
verifyHandover	bookingId, handoverPin	handoverTimestamp, status: IN_RENTAL	INVALID_PIN, PAYMENT_NOT_HELD
verifyReturn	bookingId, returnPin	returnedTimestamp, lateFineAmount, status: RETURNED	INVALID_PIN, BOOKING_NOT_ACTIVE

2. Payment Service (Main Service 2)

Operation Name	Input	Output	Errors
authorizeFunds	bookingId, rentalAmount, depositAmount, paymentToken	transactionId, authStatus: HELD	PAYMENT_DECLINED, INVALID_PAYMENT_TOKEN
captureSettlement	bookingId, deductions, platformFee	settlementId, payoutAmount, refundAmount	TRANSACTION_NOT_HELD, SETTLEMENT_MISMATCH
processRefund	bookingId, refundPercentage, reason	refundId, refundedAmount	RECORD_NOT_FOUND, ALREADY_REFUNDED

3. Catalogue Service

Operation Name	Input	Output	Errors
reserveEquipment	equipmentId, startDate, endDate	lockToken, isLocked: true	OVERLAPPING_SCHEDULE, EQUIPMENT_INACTIVE
releaseLock	lockToken	isReleased: true	TOKEN_NOT_FOUND

4. User Service

Operation Name	Input	Output	Errors
validateUser	userId	isKycVerified, accountStatus	USER_NOT_FOUND, ACCOUNT_SUSPENDED

Task 4: Central Operation Specification — `requestBooking`

Analogous to `placeOrder` in e-commerce; this operation drives the end-to-end lifecycle in AgriRent.

1. Inputs

- `renterId`: Unique identifier of the authenticated user requesting the machinery.
- `equipmentId`: Unique identifier of the selected agricultural equipment.
- `startDate`: Requested rental starting date (`YYYY-MM-DD`).
- `endDate`: Requested return date (`YYYY-MM-DD`).
- `paymentMethodToken`: Pre-authorized payment credential token.

2. Outputs

- `bookingId`: Unique UUID generated for the transaction.
- `rentalBreakdown`:
  - `totalDays`: Total calendar rental days.
  - `baseRentalFee`: Computed as `days * pricePerDay`.
  - `securityDeposit`: Held deposit amount required by the owner.
  - `platformFee`: 3% AgriRent facilitation fee.
  - `totalAuthorized`: Total amount to be held in escrow upon owner approval.
- `approvalDeadline`: ISO timestamp set to exactly 24 hours from creation.
- `status`: `PENDING_OWNER_APPROVAL`

3. Error Cases

- `400 BAD_REQUEST (INVALID_DATE_RANGE)`: If `endDate` is prior to `startDate` or outside permissible operational windows.
- `403 FORBIDDEN (KYC_REQUIRED)`: If the `renterId` has not completed mandatory identity verification.
- `404 NOT_FOUND (EQUIPMENT_NOT_FOUND)`: If `equipmentId` is non-existent, archived, or deactivated.
- `409 CONFLICT (SCHEDULE_OVERLAP)`: If the machinery is already reserved by another party for overlapping dates.

4. Encapsulation (Internal Details Hidden from Callers)

- The database engine schemas, table primary keys, and physical storage indexes.
- The 3% platform margin allocation logic and internal revenue split accounts.
- Direct contact credentials and private phone numbers of the equipment owner (masked until approval).
- Database locking primitives and distributed concurrency locks used to prevent double booking.

Task 5: Database Schema & Isolation

- **Files:** `schema.drawio`, `schema.png`, and `schema.sql`
- Complies strictly with single-ownership rules: no tables are shared across service boundaries.

Task 6: Service Property Validation

Validation of each microservice against the five fundamental architectural properties:

Service Name	Reachable	Self-Contained	Has Contract	Independent	Loosely Coupled	Architectural Assessment & Fix Note
User Service	Yes	Yes	Yes	Yes	Yes	**PASS:** Fully self-contained identity provider exposed over REST.
Catalogue Service	Yes	Yes	Yes	Yes	Yes	**PASS:** Manages listings and calendar availability independently.
Booking Service	Yes	Pass (Revised)	Yes	Yes	Pass (Revised)	**FIXED:** Initially tightly coupled to the catalog database for pricing. Decoupled by consuming explicit contract endpoints (`checkAvailability`, `reserveEquipment`).
Payment Service	Yes	Yes	Yes	Yes	Yes	**PASS:** Operates independently, interacting with financial gateways without direct database ties to other services.